

Report

Watermark removal method

Problem Statement

Let $I(x, y)$ be the given watermarked image of size $H \times W$, where $x \in [1, W]$ and $y \in [1, H]$. The goal is to remove the watermark located in a rectangular region, such that the output image $\hat{I}(x, y)$ does not contain the watermark, and the removed region is visually consistent with its surroundings.

Mask Creation

Locate the coordinates of top left corner (x_1, y_1) and bottom right corner (x_2, y_2) of rectangular watermark region.

Define a binary mask $M(x, y)$ with dimensions $H \times W$, such that:

$$M(x, y) = \begin{cases} 1, & \text{if } x_1 \leq x \leq x_2 \text{ and } y_1 \leq y \leq y_2, \\ 0, & \text{otherwise.} \end{cases}$$

Here, $M(x, y)$ identifies the region of the watermark to be removed.

Inpainting process

To remove the watermark, the region identified by $M(x, y)$ is reconstructed using the *Telea Inpainting Algorithm*. The algorithm propagates pixel values from the boundary of the mask into the masked region using both intensity and gradient information.

The inpainted image $\hat{I}(x, y)$ is computed as:

$$\hat{I}(x, y) = \text{Inpaint}(I(x, y), M(x, y), r),$$

where:

- $\text{Inpaint}(\cdot)$ is the inpainting function,

- r is the inpainting radius, defining the neighborhood size used for reconstruction.

The *Telea Algorithm* solves the inpainting problem by iteratively propagating pixel values inward from the boundary of the mask.

Output

The result is the inpainted image $\hat{I}(x, y)$, where the masked region has been replaced with visually consistent content derived from its surroundings.

Image compression and decompression pipeline

This section describes the image compression-decompression pipeline. The process includes applying the Discrete Cosine Transform (DCT), quantization, thresholding, and encoding the compressed data into a bitstream. It also details the decoding process to reconstruct the image from the bitstream.

Pipeline description

The image compression-decompression pipeline consists of the following stages:

Encoder

1. Convert the image to grayscale.
2. Apply the 2D Discrete Cosine Transform (DCT).
3. Quantize the DCT coefficients using a quantization factor q .
4. Apply thresholding to reduce sparsity.
5. Flatten the coefficients and compress them into a bitstream using BZ2 compression algorithm.

Decoder

1. Decode the compressed bitstream to retrieve quantized DCT coefficients.
2. Reshape the coefficients to their original block shape.

3. Perform dequantization using the quantization factor q .
4. Apply the Inverse Discrete Cosine Transform (IDCT) to reconstruct the image.

Pseudocode

Notation

- Let I denote the grayscale image of dimensions $M \times N$.
- Let $T = \text{DCT}(I)$ represent the 2D Discrete Cosine Transform applied to I .
- Let Q be the quantization matrix, where $Q_{ij} = q$ for all i, j .
- Let $\hat{T} = \text{round}(T/Q)$ denote the quantized coefficients.
- Let $\hat{T}_{\text{thresh}}$ denote the coefficients after thresholding, where:

$$\hat{T}_{\text{thresh}}(i, j) = \begin{cases} \hat{T}(i, j), & \text{if } |\hat{T}(i, j)| \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

- Let B represent the compressed bitstream generated from $\hat{T}_{\text{thresh}}$.
- Let $\tilde{T}$ denote the coefficients after decompression and dequantization:
 $\tilde{T} = \hat{T}_{\text{decompressed}} \cdot Q$.
- Let $\tilde{I} = \text{IDCT}(\tilde{T})$ denote the reconstructed image after applying the Inverse DCT.

Encoder

Algorithm 1 Image Compression Encoder

Require: Grayscale image I , quantization factor q , threshold τ .

Ensure: Compressed bitstream B , quantization matrix Q , and original shape of coefficients.

- 1: Compute $T \leftarrow \text{DCT}(I)$.
- 2: Generate Q such that $Q_{ij} = q$ for all i, j .
- 3: Compute quantized coefficients: $\hat{T} \leftarrow \text{round}(T/Q)$.
- 4: Apply thresholding:

$$\hat{T}_{\text{thresh}}(i, j) \leftarrow \begin{cases} \hat{T}(i, j), & \text{if } |\hat{T}(i, j)| \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

- 5: Flatten $\hat{T}_{\text{thresh}}$ into a 1D array.
 - 6: Compress the array into a bitstream B using BZ2 compression algorithm.
 - 7: **return** $B, Q, \text{shape of } \hat{T}$.
-

Decoder

Algorithm 2 Image Decompression Decoder

Require: Compressed bitstream B , quantization matrix Q , shape of original coefficients.

Ensure: Reconstructed grayscale image $\tilde{I}$.

- 1: Decompress B into a 1D array of quantized coefficients.
 - 2: Reshape the array into its original shape using the given shape.
 - 3: Compute dequantized coefficients: $\tilde{T} \leftarrow \hat{T} \cdot Q$.
 - 4: Reconstruct the image: $\tilde{I} \leftarrow \text{IDCT}(\tilde{T})$.
 - 5: Clip values of $\tilde{I}$ to range $[0, 255]$.
 - 6: **return** $\tilde{I}$.
-

Reconstructing the image from bitstream

To reconstruct the image:

- Convert the bitstream B into a 1D array of quantized coefficients.
- Use the provided quantization matrix Q and original shape to perform dequantization and reshaping.
- Apply the Inverse DCT to the dequantized coefficients to obtain the reconstructed image.

Output of image at each stage of pipeline

The change in the output of the grayscale image at the different stages of the image compression and decompression pipeline is shown in the Jupyter notebook.

Histogram of original and decompressed images

The histogram plot of the original (given image after removing watermark) and decompressed grayscale images is present in the Jupyter notebook.

Improvements to the pipeline

1. Thresholding for Sparsity

Objective: Reduce the number of nonzero DCT coefficients by applying a threshold.

Definition:

$$T_{\text{quantized}}(i, j) = \begin{cases} T(i, j), & \text{if } |T(i, j)| \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

where $T(i, j)$ represents the quantized DCT coefficient and τ is the threshold.

Impact: Increases sparsity, improving compression while maintaining visual quality.

2. Improved Quantization Factor

Objective: Adjust the quantization factor Q_f dynamically to balance compression and image quality instead of a fixed Q_f .

Quantization Rule:

$$Q(i, j) = \frac{T(i, j)}{Q_f}, \quad Q_{\text{quantized}}(i, j) = \text{round}(Q(i, j)).$$

Impact: Retains critical image details while reducing data size.

3. Entropy-Based Compression

Method: Use BZ2 compression instead of Zlib to encode sparse quantized coefficients.

Definition:

$$\mathbf{Q}_{\text{compressed}} = \text{BZ2}(\mathbf{Q}_{\text{flat}}),$$

where $\mathbf{Q}_{\text{flat}}$ is the flattened quantized coefficient matrix.

Impact: Achieves higher compression ratios for sparse data compared to Zlib.